Product Requirements Document (PRD): TiffinTrack

Project Name: TiffinTrack

Platform: Mobile Application (React Native - iOS & Android)

Target Users: Tiffin Service Customers (User App) and Kitchen Owners/Managers (Admin App)

Objective: To build an automated, transparent, and scalable system that manages daily tiffin deliveries, tracks attendance without disputes, handles sudden cancellations, and automates monthly billing.

---

1. Core Business Logic (The Rules Engine)

Before diving into features, these three absolute rules govern the entire application's backend architecture:

1. The Validation Rule (Double-Handshake): A meal is only marked as "Billable" (added to the monthly invoice) if Admin_Status == 'Sent' AND User_Status == 'Received'.

2. The Strict Cut-Off Rule (Lock-Out): The system operates on a strict binary pause system to prevent kitchen wastage. If a user pauses before the Admin's global Cut-Off Time, the meal is saved and not billed. Once the Cut-Off Time passes, the meal is locked, dispatched, and billed at 100%. There are no partial cancellation fees.

3. The Auto-Approval Rule (Forgetful User Fallback): If the Admin marks a meal as 'Sent', but the user fails to interact with the app (either to confirm receipt or raise a dispute) within 12 hours of dispatch, a background Cron Job will automatically default the user's status to 'Received'. This guarantees the meal becomes billable and protects the Admin from revenue loss due to inactive users.

---

2. User Application Requirements (Customer Side)

This interface must be highly intuitive, requiring minimal daily interaction from the user.

Feature Name	Functional Description & Workflow
1. Smart Onboarding & Profile	Users register via phone/email. They input their full name, precise delivery address, and dietary preferences (e.g., Veg, Non-Veg, Allergies). **Step 1: Secure Authentication (The Entry)** • **The Screen:** A clean login screen asking for their Mobile Number. • **The Action:** The user enters their 10-digit number and receives a 4-to-6 digit OTP (One Time Password) via SMS.

Feature Name	Functional Description & Workflow
	• **The Tech:** This is powered by Supabase Auth. It instantly verifies the number. If they are a new user, it moves them to Step 2. If they are an existing user, it drops them straight into the Main Dashboard. **Step 2: Essential Details (Who & Where)** • **The Screen:** "Welcome! Let's get your food to the right place." • **The Action:** The user enters: ○ **Full Name.** ○ **Delivery Address:** Instead of just a text box, you can provide a "Use Current Location" button (using React Native Geolocation) to auto-fill their sector/street, along with a text box for "Flat No. / Landmark." **Step 3: The "Smart" Preferences (What they eat)** This is where the app adapts to the user. • **The Screen:** "What kind of tiffin are you looking for?" • **The Action:** * **Dietary Tag:** They select **Veg** or **Non-Veg** (represented by nice visual icons). ○ **Subscription Type:** They tap on **Lunch**, **Dinner**, or **Both**. • **The "Smart" Logic:** The app remembers this choice for the next screen. If they only selected "Lunch," the app will completely hide all dinner-related questions moving forward. **Step 4: Setting the Notification Timers (The Core Hook)** • **The Screen:** "When do you usually eat?" • **The Action:** * If they selected "Lunch," a clean digital clock picker appears: *"Set your expected Lunch delivery time."* (e.g., they select 1:30 PM). ○ If they selected "Both," a second clock picker appears for Dinner (e.g., they select 8:30 PM). • **The Finish:** They hit a big "Complete Setup" button.
2. Multi-Meal Subscription	**1. The User Setup (React Native UI)** When a user is onboarding or editing their plan, they are presented with a simple choice: • **Meal Selection:** They can toggle their preference to **Lunch Only**, **Dinner Only**, or **Both**.

Feature Name	Functional Description & Workflow
	• **Combo Pricing Logic:** If the user selects both "Lunch" and "Dinner," your React Native frontend will dynamically calculate and display a discounted "Combo Price" to encourage higher sales. For example, instead of ₹100 for Lunch and ₹100 for Dinner, the UI offers both for ₹180/day. **2. Granular Meal Pausing (The Tricky Part, Solved)** If a user is subscribed to both meals, they must have the ability to manage them independently. • **Scenario:** A user has an office party for lunch but still wants their tiffin for dinner. • **The Action:** The user taps "Pause" on the Lunch card before the 10:00 AM cut-off time. • **The Result:** The backend flags *only* their Lunch as paused for that specific date. Their Dinner remains active. Because you are using a postpaid ledger, the paused lunch is simply **not added** to their monthly bill. There is no complex math to refund or restore a "quota." **3. The Admin Dispatch Queues** For the kitchen, managing multi-meal subscribers needs to be completely frictionless. • **Afternoon Dispatch:** When the Admin opens the "Live Dispatch Dashboard" at 12:30 PM and selects the **Lunch** filter, the UI populates with everyone supposed to receive a lunch today. This list automatically includes Lunch-only subscribers AND your Multi-Meal subscribers. • **Evening Dispatch:** Later in the day, when the Admin switches the filter to **Dinner**, those same Multi-Meal users will seamlessly reappear in the queue to have their second meal cooked and dispatched. **4. Database Architecture (How to Store It)** To make this work seamlessly, your users or subscriptions table should use boolean flags or an array rather than a single string. • **Example Schema:** Instead of plan: 'combo', use is_lunch_active: true and is_dinner_active: true. • **The Delivery Log:** Every single tiffin delivery must be logged as an individual row in your ledger_entries table with a specific meal_type (e.g., Lunch or Dinner) and its respective price. This gives you a crystal-clear audit trail for the 30-day invoice generation.

Feature Name	Functional Description & Workflow
3. Custom Smart Notifications	Users set an "Expected Arrival Time" for each selected meal (e.g., Lunch at 1:30 PM). A push notification triggers precisely at this time asking: *"Has your tiffin arrived?"*
4. Two-Way Confirmation	The user interacts with the notification by selecting **"Yes"** or **"No"**. • **Yes:** Validates delivery. • **No:** Opens a prompt requiring a reason (e.g., Late, Missing), which alerts the Admin immediately.
5. Strict Pause (Buffer Rule)	**Strict Cut-Off & Pause Logic (The Buffer Rule)** To prevent food wastage and protect the kitchen's revenue, the system enforces a strict time-based lock on meal cancellations. There are no partial fees; it is a binary system. • **The Core Concept:** The Admin sets a global "Buffer Time" (e.g., 3 hours before dispatch). If Lunch dispatch is 1:00 PM, the strict cut-off is 10:00 AM. • **State 1: Before Cut-Off (Free Pause):** If the user taps "Pause Meal" before 10:00 AM, the cancellation is successful. The meal is not cooked, not dispatched, and **not billed** to the user's monthly ledger. The meal is effectively saved. • **State 2: After Cut-Off (Order Locked):** At exactly 10:00 AM, the React Native UI automatically disables the "Pause" button for that day's lunch. The button turns gray and changes to: *" 🔒 Preparing in Kitchen"*. • **No Emergency Cancellations:** If the user tries to cancel after the cut-off, the app strictly blocks it. The food will be dispatched, and the user will be billed for the meal (100% charge) because the kitchen has already utilized the raw materials. **3. The Backend Logic (The Security Layer)** Since you are handling the backend, you know you can never trust the client's device time (a user could easily change their phone's clock to bypass the rules).

Feature Name	Functional Description & Workflow
	• **Server-Side Validation:** When the React Native app sends the POST /cancel-meal request, the backend immediately checks the server's current timestamp. **Real-time Sync:** The backend should send the exact cut-off timestamps to the frontend when the app loads, so the UI countdowns are perfectly synced with the server. **4. The Admin & Kitchen Operations** This feature directly ties into the "Bulk Mark Sent" feature we discussed earlier. • **The Locked List:** At exactly 10:00 AM, the backend "locks" the preparation list. The admin's dashboard updates to show: *"Final Lunch Count: 45 Tiffins."*
6. Vacation Mode	An interactive calendar where users select a date range (e.g., Oct 10 to Oct 15) to bulk-pause deliveries. Notifications and billing are suspended for these dates. "Vacation Mode" is an absolute must-have for TiffinTrack. Since your target audience likely includes students and working professionals, they will frequently travel home for long weekends or holidays. Without this feature, a user would have to manually open the app every single morning to hit "Pause," which is annoying for them and unpredictable for your kitchen. Here is exactly how you can architect "Vacation Mode" across your full-stack setup, making it seamless for the user and fully automated for the admin. **1. The Core Concept (Date Range vs. Daily Pause)** While your standard "Pause" is for a single meal or day, **Vacation Mode** uses a definitive **Start Date** and **End Date**. During this period, the system completely freezes their subscription. **2. The User App Experience (React Native UI)** You want to make this highly visual so users feel confident their food won't be wasted. • **The Calendar Picker:** Use a library like react-native-calendars to let the user select a date range. • **The Visual Confirmation:** Once selected, the user's dashboard should change its primary state. Instead of showing "Today's Menu," it should display a large banner: ✈️ *"On Vacation until April 5th. Enjoy your trip!"*

Feature Name	Functional Description & Workflow

	• **Resume Early Button:** Plans change. Always include an "I'm Back Early – Resume Tiffins" button just in case they return before their selected end date.
	
	3. The Database Schema (How to Store It)
	
	Instead of creating a massive list of paused days, you just need to store the date ranges in your database.
	
	In your User or Subscription table, add an array field called vacation_periods:
	
	JSON
	
	"vacation_periods": [
	
	{
	
	"start_date": "2026-04-01",
	
	"end_date": "2026-04-05"
	
	}
	
	]
	
	4. The Backend Logic (The Dispatch Filter)
	
	This is where the magic happens. You don't need a Cron Job to constantly check who is on vacation; you just need to update your **Dispatch Query**.
	
	When the admin clicks "Bulk Mark Sent" at 12:00 PM, your backend fetches the list of users to deliver to. You simply add one condition to your database query:
	
	• **The Rule:** Fetch all active users WHERE today's date does NOT fall between any start_date and end_date in their vacation_periods array.
	
	• **The Result:** The backend completely ignores these users. They don't show up on the kitchen's prep list, and they don't get a delivery boy assigned.
	
	Dynamic Billing (No Quotas): Because the app uses a Postpaid Ledger, taking a vacation doesn't require extending a 'plan'. The days are simply skipped, and the user's invoice at the end of their 30-day cycle will only reflect the exact number of meals they actually received."

7. Daily Menu Preview	A section on the home screen displaying the day's specific menu (updated by Admin) to help users decide whether to consume or cancel that day's meal. **1. The User Experience (React Native UI)**

Feature Name	Functional Description & Workflow
	The menu should be the very first thing the user sees when they open the app. • **The Dashboard Card:** On the home screen, display a beautifully styled card for "Today's Lunch" and "Today's Dinner" (tying perfectly into your Multi-Meal feature). • **Visual Tags:** Display the menu as a clean list with emojis or tags. ○ *Example:* 🫓 4 Butter Rotis | 🥣 Dal Tadka | 🍛 Paneer Bhurji | 🥗 Salad • **The "Karela" (Bitter Gourd) Integration:** Place the "Pause Meal" button directly below the menu preview. If a user sees a dish they don't like (like Karela), they can instantly hit Pause. This ties perfectly into the **Strict Pause** logic we discussed earlier—if they see the menu early enough, they can skip it without penalty! **2. The Admin Experience (Data Entry)** Tiffin business owners are busy cooking; they don't have time for complex data entry. You need to make updating the menu frictionless. • **The Weekly Template:** Most tiffin services have a set weekly menu (e.g., Monday is always Rajma Chawal). The admin dashboard should allow them to set a "Default Weekly Menu" that automatically repeats. • **Daily Overrides:** The admin should have a simple text input screen where they can select a date on a calendar and quickly type out a custom menu if they decide to change things up at the last minute. • **Push Notification Trigger:** Once the admin publishes the menu for the next day, a button should allow them to blast a notification: "🔔 *Tomorrow's Lunch Menu is updated! Check the app.*" **3. The Database Schema (Backend Architecture)** To make this efficient, you should create a dedicated table just for menus, rather than attaching the menu to individual user records. **Table Name:** daily_menus • id: UUID (Primary Key) • target_date: Date (e.g., "2026-03-28") • meal_period: Enum (Lunch, Dinner) • menu_items: Array of Strings (e.g., ["4 Roti", "Aloo Gobi", "Jeera Rice", "Papad"])

Feature Name	Functional Description & Workflow
	• is_special: Boolean (To highlight special Sunday or festival meals with a star icon in the UI). **The API Fetch Logic:** When the user's React Native app opens, it makes a simple GET /api/menu?date=today request. Because this data is identical for all users, you can heavily cache this API response using something like Redis to save database read costs if your app scales up. **4. Handling Edge Cases & Business Rules** • **Last-Minute Kitchen Changes:** Sometimes the kitchen runs out of Paneer and has to make Chana Masala instead. If the admin edits the menu *after* 9:00 AM, the backend should automatically trigger a push notification to users alerting them of the substitution. • **Dietary Flags:** In the future, you could add simple boolean tags to the database schema like is_spicy: true or contains_dairy: true so the React Native UI can display warning icons for users with allergies or preferences.
8. Ledger & In-App Payments	A monthly visual calendar showing Green (Received), Yellow (Late Cancel), and Red (Paused) days. Month-end auto-calculates the bill. Includes a "Pay Now" button linked to UPI/Payment Gateways. **1. The Core Concept (The Dynamic Khata)** The cycle starts on "Day 1." For the next 30 days, the user receives their tiffins. Every time a tiffin is successfully marked as Delivered (via the Admin or Photo Proof), an entry is added to their ledger. On "Day 30," the app automatically tallies up only the delivered meals, generates an invoice, and asks the user to pay. **2. The User App Experience (React Native UI)** The user should always know exactly what their upcoming bill looks like so there is no "sticker shock" at the end of the month. • **The Running Ledger (Days 1–29):** On their dashboard, display a "Current Month Khata" card. It should show a live count: *"18 Meals Delivered. Estimated Bill: ₹1,800."* Clicking this opens a detailed list showing exactly which dates they received food and which dates were paused. • **The Bill Generation (Day 30):** On the 30th day, the dashboard transforms. The running total disappears and is replaced by a bold Invoice Card: *"Your 30-Day Cycle is Complete. Total Bill: ₹2,600."* * **The "Pay Now" Button:** Below the invoice is a prominent button to trigger the UPI or Razorpay payment. **3. Backend & Database Architecture**

Feature Name	Functional Description & Workflow
	To make this work securely, you need to separate the *daily logs* from the *monthly invoices*. **Table 1: ledger_entries (The Daily Tracker)** Every successful delivery creates a row here. • id: UUID • user_id: Foreign Key • date: Date • meal_type: Enum (Lunch, Dinner) • amount: Decimal (e.g., ₹100) • status: Enum (UNBILLED, BILLED) **Table 2: invoices (The 30-Day Bill)** • invoice_id: UUID • user_id: Foreign Key • cycle_start_date: Date • cycle_end_date: Date • total_amount: Decimal • status: Enum (UNPAID, PAID) **4. The "Day 30" Automation (The Cron Job)** You do not want the admin to manually calculate bills. You will set up a background task (a Cron Job) on your Node.js or Python backend that runs every night at midnight. 1. The script finds all users whose 30-day cycle ends today. 2. It queries ledger_entries to sum up all UNBILLED meals for that user. 3. It creates a new record in the invoices table with the total_amount. 4. It updates those specific ledger_entries from UNBILLED to BILLED so they aren't charged twice. 5. It sends a push notification: *"Your monthly tiffin bill is generated! Tap to pay."* **5. Handling Payments & Grace Periods** When dealing with postpaid models, you must have a system for late payments.

Feature Name	Functional Description & Workflow
	• **The Grace Period:** When the bill is generated on Day 30, give the user a 2 to 3-day grace period. During this time, the kitchen continues to send food for the *new* cycle. • **The Auto-Lock (Admin Protection):** If the invoice remains UNPAID by Day 33, the backend automatically changes the user's subscription status to SUSPENDED. They disappear from the admin's daily dispatch list, and the app displays a locked screen: *"Please clear your previous dues of ₹2,600 to resume your daily tiffins."* • **Payment Completion:** Once the user taps "Pay Now" and the UPI/Razorpay webhook confirms the transaction, the invoice status changes to PAID, the suspension is instantly lifted, and the cycle resets.

---

3. Admin Application Requirements (Provider Side)

This interface is an operational dashboard designed for speed, overview, and financial control.

Feature Name	Functional Description & Workflow
1. User & Rate Management	A master directory of all customers. The Admin can approve new users, edit details, and set a **custom per-meal rate** for specific individuals. **1. The Core Concept (Dynamic Rate Cards)** You must **never hardcode** the price of a tiffin in your frontend or backend code. Prices change due to inflation, and admins need to create custom deals. Instead, you will create a system of **"Rate Cards"** (or Pricing Tiers) that the admin can assign to individual users. **2. The Admin Experience (Dashboard UI)** When the admin opens the "Customers" tab on their dashboard, they need powerful but simple controls. • **The User Directory:** A searchable, filterable list of all customers. Filters should include: *Active*, *On Vacation*, *Suspended (Unpaid)*, and *By Route/Zone*. • **The Onboarding Form:** When adding a new user, the admin fills out: 1. **Basic Info:** Name, WhatsApp Number. 2. **Logistics:** Delivery Address, GPS Pin (critical for the delivery boy app), and **Route/Zone** (e.g., "Koramangala - Route A").

Feature Name	Functional Description & Workflow
	3. **Dietary Tags:** Simple toggles for *Veg*, *Non-Veg*, *Jain*, or *Less Spicy*. 4. **Rate Assignment:** A dropdown menu to select their specific pricing tier (e.g., "Student Rate"). • **Manual Overrides:** The admin must have the ability to click on a user's profile and say, *"Give this specific person a ₹500 flat discount this month"* (adding a manual credit to their ledger). **Backend & Database Architecture** To make this highly scalable, you need a clean relational database structure. Here are the three primary tables involved: **Table 1: rate_tiers** • id: UUID • tier_name: String (e.g., "Student Veg") • price_per_meal: Decimal (e.g., 80.00) **Table 2: delivery_zones** • id: UUID • zone_name: String (e.g., "North Campus") • daily_delivery_fee: Decimal (e.g., 10.00) **Table 3: users** • user_id: UUID (Primary Key) • name: String • phone: String • rate_tier_id: Foreign Key (Links to rate_tiers) • zone_id: Foreign Key (Links to delivery_zones) • status: Enum (ACTIVE, PAUSED, SUSPENDED) • dietary_preferences: Array of Strings (['veg', 'no_onion'])
2. Live Dispatch Dashboard	A real-time counter that automatically deducts users on "Vacation Mode" or who used " Free Pause " It displays the exact number of meals to prepare: *"Cook Today: 45 Lunches, 38 Dinners."*
3. "Mark as Sent" & Override	Admin toggles a button to mark meals as "Sent" when they leave the kitchen. Admin can also manually force a "Received" status for a user, which instantly sends an SMS/Alert to the user for transparency.

Feature Name	Functional Description & Workflow
	1. "Mark as Sent" (Normal Daily Flow) This is used when the tiffin is packed and handed over to the delivery boy. • **The UI (How it looks):** The Admin app displays a dashboard list of all active users expecting a meal that day. Next to each user's name is a Toggle Switch. • **The Action:** As soon as a tiffin is dispatched, the Admin flips the toggle **ON** (it turns from gray to green). • **The Backend Magic:** ○ The database updates that day's entry, changing admin_status to 'sent'. ○ The system instantly fires a Push Notification to the Customer's phone: *"Your tiffin is on the way!"* • **The User's Duty:** It is now up to the user to tap **"Yes, Received"** in their app once the food arrives. This changes the user_status to 'received' in the database, officially adding the charge to their monthly bill.
	2. "Manual Override" (The Exception Flow) Now, consider a practical problem: The user received the tiffin and ate the food, but they simply forgot to open the app and tap "Yes, Received." Or perhaps their phone battery died. If the user never presses "Yes," the system won't add that meal to the final bill, and the Admin loses money. The **Override** function solves this. • **Keep the 12-hour Auto-Approval Cron job for forgetful users. Reserve the "Manual Override" *only* for resolving active disputes (e.g., the user clicks "No, I didn't get it", but the Admin has photo proof that they did)** • **The Backend Magic:** The database forcefully updates the user_status to 'received', ensuring the meal is counted toward the final billing cycle.
	3. The "Anti-Fraud" Security (Dispute Prevention) There is a massive risk with the Override function: *What if a dishonest Admin uses the Override button to charge users without actually sending them food?*

Feature Name	Functional Description & Workflow
	To prevent fraud and disputes, we implement a strict security logic: • **The Trigger:** The exact second the Admin taps the "Override" button, the TiffinTrack system bypasses the app and sends a standard **Text SMS** (or automated WhatsApp message) directly to the user's phone number. • **The Message:** *"Alert from TiffinTrack: The Admin has manually marked today's tiffin as 'Delivered'. If you did not receive your food, please open the app and raise a 'Dispute' within the next 12 hours."*
4. Live Delivery Tracking	Instead of the delivery boy logging into an app, the Admin generates a simple, temporary web link that the delivery boy opens in their phone's standard web browser (like Chrome or Safari). **Step 1: The Admin Action (Generating the Link)** • **The UI:** On the Admin's DispatchDashboard.js, right next to the "Mark Route as Sent" button, there is a button labeled **"Generate Tracking Link."** • **The Backend:** Your Node.js/Supabase backend generates a temporary, unique URL (e.g., tiffintrack.com/track/route-A-today). This URL is valid for only 3 hours. • **The Handoff:** The Admin taps "Share Link" and sends it to the delivery boy via WhatsApp before they leave the kitchen. **Step 2: The Delivery Boy Action (The Broadcaster)** • **No App Install:** The delivery boy clicks the link on WhatsApp. It opens a very basic, single-page website. • **Permissions:** The webpage asks: *"TiffinTrack wants to use your location. [Allow]"* • **The Script:** Once allowed, a simple JavaScript function (navigator.geolocation.watchPosition) runs in their browser. Every 15 seconds, it silently pings their latitude and longitude to your Supabase Realtime channel. • **The UI for the Boy:** The webpage just shows a big green screen saying: *"Tracking Active. Please keep this tab open while delivering."* **Step 3: The Admin Interface (The Receiver)** This is where your requested feature lives. • **The Map Screen:** The Admin opens LiveTrackingMap.js in their app. • **The Listener:** The Admin app subscribes to that exact Supabase Realtime channel.

Feature Name	Functional Description & Workflow
	• **The Visuals:** As the browser on the delivery boy's phone pings the coordinates, the React Native Map updates the marker's state, showing the Admin exactly where the food is in real-time. • **The Kill Switch:** Once the Admin confirms all tiffins are delivered (or 3 hours pass), the temporary link expires. If the delivery boy keeps the browser open, it will simply say "Session Ended," protecting their privacy.
5. Global Settings Control	Admin interface to configure system-wide variables, primarily the **Cut-off Times** for Lunch and Dinner cancellations. **1. The Admin Dashboard (The Master Switches)** On the admin web panel, create a dedicated "App Settings" page divided into logical categories. The admin can tweak these at any time: • 🛑 **Operational Toggles (Emergency Stops):** ○ accepting_new_subscriptions: [ON/OFF] (Turn off if the kitchen is at maximum capacity). ○ kitchen_active_today: [ON/OFF] (Turn off during extreme weather or emergencies to instantly pause all deliveries). • ⏱️ **Time & Buffer Rules:** ○ lunch_dispatch_time: 12:30 PM ○ pause_buffer_hours: 3 • 💰 **Financial Controls:** ○ default_delivery_fee: ₹15 ○ max_negative_wallet_balance: -₹500 (Ties into our 30-Day Postpaid logic). • 📢 **Dynamic Content & Support:** ○ customer_support_whatsapp: +91-9876543210 ○ app_marquee_banner: " 🎉 Happy Diwali! Please note, the kitchen will be closed for Dinner on 12th Nov." **2. The User App Experience (React Native Impact)** The React Native app acts as a mirror to these settings. • **The Splash Screen Load:** When the user opens the app, the very first thing React Native does (while showing the loading logo) is make an API call to GET /api/settings/global.

Feature Name	Functional Description & Workflow
	• **Global State:** The app saves this response into its global state management (like Redux, Zustand, or React Context). • **Instant UI Changes:** * If kitchen_active_today is False, the home screen immediately locks and displays: " 🎉 *Kitchen is closed today due to heavy rain. All meals are paused and will not be billed.*" ○ The "Contact Support" button dynamically opens WhatsApp using the customer_support_whatsapp variable, meaning if the admin changes their phone number, the app updates instantly without an app store release. **3. Backend & Database Architecture** Since this data changes rarely but is read constantly (every time any user opens the app), you need a specific database strategy. **The Database Schema (global_settings table):** Instead of making a table with 50 different columns, the best practice is a **Key-Value** structure. • setting_key: String (Primary Key) - e.g., lunch_buffer_time • setting_value: String/JSON - e.g., "3" • data_type: Enum (boolean, number, string, json) • last_updated: Timestamp **The Performance Hack (Caching):** Because every single user queries these settings on app load, reading from the database every time is inefficient. • Your backend should load these settings into server memory or a Redis cache. • When the React Native app asks for the settings, the backend serves them from the cache instantly (taking 10ms instead of 100ms). • When the Admin clicks "Save Settings," the backend updates the database *and* refreshes the cache. **4. A Real-World Scenario: The "Rainstorm"** Imagine it is 10:00 AM, and a massive rainstorm floods the city. Delivery boys cannot go out. Without Global Settings, the admin would have to manually message 200 users and manually refund 200 ledgers. With Global Settings, the admin simply flips kitchen_active_today to **OFF**. Instantly, the React Native app locks, users see a polite warning banner, the daily billing Cron Job skips today, and the kitchen avoids a customer service nightmare.
6. Route Optimization	The app generates a daily delivery list sorted by area. Each address features a direct link that opens Google Maps navigation for the delivery boy.

Feature Name	Functional Description & Workflow
7. Automated Analytics & Billing	At the end of the billing cycle, the system calculates (Validated Meals * Rate) and generates PDF invoices. A separate "Dispute" tab flags all users who clicked "No," requiring Admin resolution. **1. The Automated Billing Engine (The "Dunning" System)** We touched on the 30-Day invoice generation earlier, but a true billing system also handles the *collection* of money automatically. This is called a "Dunning" process. • **The Cron Job Schedule:** ○ **Day 30 (12:01 AM):** The system generates the invoice, updates the ledger, and sends the first push notification: *"Your monthly bill of ₹2,400 is ready."* ○ **Day 32 (Morning):** If the status is still UNPAID, the system sends an automated "Gentle Reminder" via WhatsApp API or Push Notification. ○ **Day 35 (Midnight Cut-off):** If still UNPAID, the system triggers the Auto-Suspend protocol. It changes the user's status to SUSPENDED, removes them from the kitchen's prep list, and locks their app screen until the payment gateway clears the due amount. • **Zero Human Intervention:** The admin never has to awkwardly ask customers for money. The system acts as the strict, neutral accountant. **2. Kitchen Forecasting Analytics (The Money Saver)** Food waste is the #1 reason local food businesses fail. Your system has all the data needed to prevent this. • **The 7-Day Forecast:** On the admin dashboard, create a chart that looks 7 days into the future. • **The Formula:** Total Active Users - Users on Vacation - Predicted Pauses (based on historical data). • **The Result:** The system tells the admin: *"You need to buy groceries for exactly 42 Lunches on Monday and 48 on Tuesday."* This stops the kitchen from over-purchasing raw materials. **3. The Admin Analytics Dashboard (Business Health)** The admin web panel or app should feature a dedicated "Insights" tab with visual charts (using a library like react-native-svg-charts or Chart.js for the web).

Feature Name	Functional Description & Workflow

	• **MRR (Monthly Recurring Revenue):** A real-time tracker showing guaranteed income based on currently active 30-day subscriptions.
	• **The "Karela Index" (Menu Popularity):** The backend tracks which menu items trigger the most "Pause" actions. If the dashboard shows that 40% of users paused their meal on "Tinda (Apple Gourd) Day," the admin knows to permanently remove that dish from the menu to save revenue.
	• **Customer Churn Rate:** A simple metric showing how many users deleted the app or canceled their subscription this month versus how many new users joined.
	4. Backend Architecture (The Aggregation Pipeline)
	Calculating analytics in real-time by scanning thousands of delivery logs every time the admin opens the dashboard will crash your server. You need an optimized data strategy.
	The analytics_snapshots Table: Instead of running heavy math on the fly, create a daily Cron job that runs at 2:00 AM.
	• It calculates the day's total revenue, total deliveries, total pauses, and active users.
	• It saves this as a single row in an analytics_snapshots table.
	• When the admin opens their dashboard, the frontend just fetches the last 30 rows from this table to draw the charts instantly.
	Example Snapshot Row:
	JSON
	{
	"date": "2026-03-26",
	"active_users": 150,
	"meals_delivered": 142,
	"meals_paused": 8,
	"daily_revenue": 14200,
	"new_signups": 3
	}
	This feature turns TiffinTrack from a simple delivery tracker into a powerful business management suite.

4. Proposed Technical Stack (Updated)

- **Frontend (Mobile):** React Native (Expo/CLI)
- **Backend as a Service (BaaS):** Supabase
    - **Database:** PostgreSQL (Relational tables for Users, Subscriptions, Daily Logs).
    - **Authentication:** Supabase Auth (OTP via SMS or Email/Password).
    - **Real-time:** Supabase Realtime API (for instant updates on the Admin Dispatch Dashboard).
    - **Storage:** Supabase Storage (for storing delivery photo proofs if needed later).
- **Push Notifications:** Firebase Cloud Messaging (FCM) or Expo Push Notifications.
- **Location/Maps:** Google Maps API (Admin app only).
- **Payments:** Razorpay, or direct UPI Deep-linking.

2. React Native Folder Structure

```
TiffinTrack/
│
├── src/
│   ├── assets/            # Images, icons, and custom fonts
│   │
│   ├── components/        # Reusable UI components (used across the app)
│   │   ├── PrimaryButton.js
│   │   ├── StatusBadge.js
│   │   └── TiffinCard.js
│   │
│   ├── context/           # Global State Management
│   │   ├── AuthContext.js     # Manages user session (Supabase Auth)
│   │   └── AppContext.js      # Manages global variables (e.g., current cut-off times)
│   │
│   ├── navigation/        # React Navigation setup
│   │   ├── AppNavigator.js    # Decides whether to show Admin, Customer, or Auth stack
```

```
│   │   ├── AdminStack.js      # Admin specific routes
│   │   ├── CustomerStack.js   # Customer specific routes
│   │   └── AuthStack.js       # Login/Signup screens
│   │
│   ├── screens/               # Main UI screens
│   │   ├── Auth/
│   │   │   ├── LoginScreen.js
│   │   │   └── RegisterScreen.js
│   │   │
│   │   ├── Customer/          # User-Facing App
│   │   │   ├── CustomerDashboard.js  # Today's menu and fast actions
│   │   │   ├── MonthlyLedger.js      # Calendar and billing view
│   │   │   └── VacationMode.js       # Calendar for bulk pause
│   │   │
│   │   └── Admin/             # Provider-Facing App
│   │       ├── DispatchDashboard.js  # Live counts and "Mark as Sent" toggles
│   │       ├── UserManagement.js     # List of users to set rates
│   │       └── LiveTrackingMap.js    # Google Maps integration
│   │
│   ├── services/              # API and Database calls
│   │   ├── supabase.js        # Supabase client initialization
│   │   ├── authService.js     # Login/Signup logic
│   │   └── deliveryService.js  # Logic for Double-Handshake & updates
│   │
│   └── utils/                 # Helper functions
│       ├── constants.js       # Colors, API keys
│       └── timeFormatter.js   # Logic for comparing times (for the Cut-off rule)
│
├── app.json                   # Expo config
├── babel.config.js
```

```
└── App.js              # Root component wrapping contexts & navigators
```